

PL/PGSQL

Fonctions et Procédures

Définition d'une fonction

```
CREATE [OR REPLACE] FUNCTION nom_fonction(arguments)
RETURNS type AS $$
DECLARE
    var1 type;
BEGIN
    -- Code de la fonction
    RETURN valeur_a_retourner;
END;
$$ LANGUAGE plpgsql;
```

Fonction : retourne une valeur

Procédure : ne retourne rien

Appel d'une fonction

```
SELECT nom_fonction(arguments);
```

Définition d'une procédure

```
CREATE OR REPLACE PROCEDURE nom_procedure(arguments)
AS $$
DECLARE
    var1 type;
BEGIN
    -- Code de la procédure
END;
$$ LANGUAGE plpgsql;
```

Appel d'une procédure

```
CALL nom_procedure(arguments);
```

SELECT → retourne quelque chose | **CALL** → ne retourne rien

Passage de paramètres

```
-- Déclaration avec paramètres
CREATE FUNCTION nom_fonction(nom_var1 type, nom_var2
type)
RETURNS type AS $$ ...

-- Appel
SELECT nom_fonction('valeur1', 'valeur2');
```

Surcharge et suppression

Surcharge :

```
CREATE FUNCTION recherche(id_personne integer)
RETURNS ...;
CREATE FUNCTION recherche(nom_personne char, prenom char)
RETURNS ...;
```

Suppression :

```
DROP FUNCTION nom_fonction(arguments);
```

Variables et structures de données

Déclaration et affectation

```
-- Initialisation
nom_variable type := valeur;

-- Opération
c := a + b;

-- Requête (INTO)
SELECT id FROM table WHERE nom = 'Einstein' INTO var;

-- Multiple colonnes
SELECT nom, prenom FROM user WHERE id = 4 INTO nom, prenom;
```

Accès

```
RETURN nom_variable;  -- Nom direct, sans $ ou %
```

Constantes

```
une_constante CONSTANT integer := 4;
```

Concaténation et transtypage

```
res := 'résultat' || ' final';  
valeur::type          -- ou CAST(valeur AS type)
```

Structures de contrôle

Conditions IF

```
-- Simple  
IF condition THEN  
    instruction;  
END IF;  
  
-- Avec ELSE  
IF condition THEN  
    instruction;  
ELSE  
    instruction;  
END IF;  
  
-- Avec ELSIF  
IF condition1 THEN  
    instruction;  
ELSIF condition2 THEN  
    instruction;  
ELSE  
    instruction;  
END IF;
```

Opérateurs : < > <= >= = <> AND OR IS NULL/IS NOT NULL

Boucles WHILE

```
WHILE condition LOOP  
    instruction;  
END LOOP;
```

Boucles FOR

```
FOR var IN [REVERSE] 1..10 LOOP  
    instruction; -- var auto-déclarée  
END LOOP;
```